WRITTEN RESPONSES

Tasks 2–4 | Akilesh K R

Task 2: Understanding Agentic AI and Workflow AI

1. Agentic AI

I think of Agentic AI as giving an AI system a goal and letting it work out the steps. It can use tools like APIs or databases, check the result and decide what to do next. A chatbot mainly gives you an answer; an agent can also take action and adjust when something does not work.

That flexibility is useful, but it needs limits. I would control what the agent can access or change, and keep a person involved before any high-impact action.

2. Workflow AI

Workflow AI is more like a fixed sequence of steps. A customer message might come in, get classified, have its key details extracted and then receive a draft reply. Because the steps are known, it is generally easier to test.

I don't think one approach is always better. For a repeated task with clear rules, a workflow is usually simpler and more predictable. If the next step depends on what the system discovers, an agent may make more sense. Honestly, I would start with the simpler workflow and add agent-like decisions only when there is a clear reason.

3. AI Copilots and Assistants

To me, a copilot is useful when it helps someone while they are doing a task. It might explain a result, suggest what to check next or prepare a draft, but the person still makes the decision. An assistant can answer direct questions too; a copilot is usually closer to the work already happening.

For engineering work, I think this difference matters: AI can help an engineer understand information faster, but it should not silently make important engineering decisions.

4. Standalone Model vs AI Inside a Product Workflow

A standalone model takes an input and returns an output. That is a good way to test an idea, but it is only one part of a working product.

In practice, the application also has to find the right data, respond to events, pass results to other systems and deal with failures. It needs checks, logs, access controls and a way for someone to review important outputs.

My projects have made this clear to me: getting a model to answer correctly is not the whole job. The surrounding application has to work reliably too.

5. My Experience

I studied mechanical engineering and got interested in AI/ML through my own learning and project work. I learn best by building something, testing it and then figuring out what needs to change.

Predictive Maintenance Copilot

For this project, I used more than 163K NASA CMAPSS engine records to train an XGBoost model to predict Remaining Useful Life (RUL). The R^2 went from 0.68 to around 0.90 after I revisited the feature engineering. My first version was not capturing the useful patterns in the engine data well enough, so I had to rethink the features rather than just keep changing the model.

I then connected an LLM assistant using LangGraph and ChromaDB. With RAG, it retrieves relevant information to help explain possible engine risks. It can also create a maintenance ticket when the predicted risk crosses a threshold. The project helped me understand how a conventional ML model and an LLM assistant can do different jobs in the same application.

Factory Line Copilot

I built a local agent with LangGraph and Ollama so users could ask questions about production data in plain English. It queries the database in read-only mode, so it cannot change production records. I also built a Power BI dashboard to track OEE, using SQL on 10,966 production records.

What I found was that running a model locally involved trade-offs between privacy, speed and answer quality. I chose Ollama because I wanted to experiment with a local setup and keep the production data within the environment, rather than send it to an external model service. It also made me pay attention to query permissions and to checking whether the agent's answers matched the underlying data.

RefundRadar: Workflow Automation and Product Integration

For RefundRadar, I used n8n to receive customer messages through a webhook. Gemini identifies the message's intent and urgency and prepares a reply draft. If the result is unclear, the workflow sends it for manual review instead of guessing.

I also added retry handling for failed API calls, Telegram notifications for approvals and a Google Sheets log of approval details. Separately, I built a FastAPI refund-risk service and wrote pytest tests for it. This gave me experience connecting AI logic to an application and its surrounding workflow.

Across these projects, the main lesson for me is that the model is only one part. Data access, testing, failure handling and human review matter just as much.

Task 3: AI Use Case Evaluation

I have not used a commercial simulation platform in a work setting yet. These ideas come from my mechanical engineering background and the AI projects I have built. I see them as starting points to discuss and test with simulation engineers, not features that should be deployed without validation.

1. Material Selection Assistant using RAG

Material selection involves comparing properties such as strength, thermal conductivity, operating temperature and cost. The information is often spread across datasheets and standards, and even a small unit mismatch can lead to a wrong comparison. For example, a tool should catch the difference between conductivity reported in W/m·K and a value entered in a different unit before ranking materials.

I would build this around RAG over approved datasheets and standards, with engineering rules to filter out materials that fail required limits. The LLM would explain the shortlist and point to the source for each property, rather than inventing values.

The useful test would be a set of around 30–40 engineering questions. I would check whether suitable materials appear in the top three and verify each quoted property against the original source. If the source is missing or unclear, the assistant should say so.

2. Input Validation and Missing Parameter Detection

A simulation can fail—or worse, produce a plausible-looking but misleading result—because a boundary condition is missing, a unit is wrong or a material property has not been entered.

I would start with deterministic checks for required fields, units and valid ranges. An ML model could later flag unusual combinations, while an LLM explains the warning in plain language. I would not trust an LLM on its own to decide whether an engineering input is valid.

To test it, I would take working simulation files, deliberately introduce common mistakes and measure how many the tool catches. False alarms matter too; engineers will stop trusting it if it keeps flagging valid inputs.

3. Simulation Copilot for Results and Run History

Engineers may have dozens of simulation runs to compare. A copilot could answer questions such as, “Which run had the highest temperature?” or “What changed when the flow rate was increased?”

The agent could turn questions into read-only database queries over run history, input parameters, results and status. This is the closest of these ideas to my Factory Line Copilot work.

I would prepare 50 questions with known answers and compare the responses with the source records. I would also test vague questions and make sure the agent cannot update or delete simulation data.

4. Digital Twin for Live Thermal Prediction

A detailed thermal simulation can take time. When equipment is operating, an engineer may need a quick estimate while waiting for a full simulation or test result. A model trained on previous simulation cases could provide that faster estimate.

I would begin with inputs such as temperature, flow rate and power, along with previous simulation outputs. XGBoost would be a sensible first model to benchmark; I would only move to a neural network if the amount and structure of the data justified the extra complexity. An LLM could explain the prediction, but it should not calculate the temperature.

The important check is whether it works beyond the cases it has seen. I would validate it on simulation cases held out from training and, where possible, real sensor readings. It should flag conditions outside its training range instead of presenting an unreliable estimate as certain.

5. Result Summary and Sanity Check

A solver can finish successfully and still produce a result that does not make physical sense. A short review of key values could help an engineer spot something worth checking before using the result.

I would combine simple engineering checks—units, heat balance, mass balance and physical limits—with anomaly detection for unusual results. The LLM's role would be to turn those findings into a readable summary, with the values and warnings visible.

I would test this using known-good cases and cases where I deliberately introduce issues, such as incorrect units or reversed inlet conditions. An experienced engineer should review the warnings before the tool is relied on.

Task 4: Motivation and Ownership

1. Why do you want to join Expert Thermal / XThermal?

My mechanical engineering studies are what drew me towards thermal engineering. Heat transfer and thermodynamics were not just subjects to pass; they helped me understand why temperature affects the performance, efficiency and life of real equipment. I am particularly interested in applying that understanding to EV systems, high-power electronics, AI hardware and data centre cooling.

As power density increases, removing heat safely becomes a real design challenge. Expert Thermal's mix of simulation, testing and validation appeals to me because it connects engineering analysis with how a system behaves in practice. I want to contribute my mechanical engineering foundation and AI skills while learning the thermal workflow from experienced engineers.

I graduated with a gold medal in mechanical engineering and a 9.58 CGPA. That academic foundation is important to me, but I am now keen to apply what I learned to practical engineering problems.

2. What are you most interested in building in AI/ML?

I want to build practical tools that help thermal engineers spend less time on repetitive checks and get useful information sooner. The areas that interest me most are simulation input checks, highlighting results that look unusual and making quick temperature estimates when a full simulation is not immediately available.

My Predictive Maintenance Copilot gave me a first experience of combining prediction with explanation. I trained an XGBoost model on NASA CMAPSS data to estimate Remaining Useful Life and improved the R^2 from 0.68 to around 0.90 through feature engineering. I then added an LLM assistant to explain possible risks. I would like to explore that combination for thermal engineering, with proper validation of the engineering outputs.

3. ML models, GenAI workflows, physics-aware AI or product AI integration?

I am most interested in product AI integration because I enjoy connecting models and workflows to solve a practical problem. RefundRadar showed me that the model is only one part of the system: API failures, retries, manual review and testing all affect whether someone can actually use it.

Physics-aware AI interests me as well. My mechanical engineering background gives me a starting point in energy balances and boundary conditions. I would like to learn how to use those constraints alongside ML so predictions make physical sense, not just a good score on a test set.

4. How do you handle ambiguous problems?

When a problem is unclear, I first separate what is known from what is missing. Then I choose a small first result I can test, make my assumptions clear and build from there. I try not to overcomplicate the first version.

That is how I approached Factory Line Copilot. I began with a few questions about production data, checked whether the agent could answer them and kept database access read-only. I would take the same approach in thermal work: start with basic calculations and clear assumptions, then improve the solution with feedback.

5. Are you comfortable working in a fast-moving startup environment?

I have not worked in a startup yet, so I know there will be a learning curve. But my project work has involved learning tools independently, working through problems and deciding what to try next without having every step laid out.

I learned LangGraph, ChromaDB, n8n, FastAPI and Ollama while building those projects. Managing them alongside my studies also taught me to plan my time. I am comfortable with changing priorities and taking responsibility for the work assigned to me.

6. How do you think you can grow within our team?

I would first learn how the products and thermal team work: how simulations are set up, how results are reviewed and how they are checked against test data. I think it is important to understand the real workflow before proposing an AI feature.

Once I understand that, I would look for a small problem, build a prototype and test it with engineers. Over time, I want to improve my thermal knowledge and become capable of taking an AI feature from an initial idea to a reliable tool.

7. Ownership, commitment and initiative

To me, ownership means not stopping when the first version works. It means checking the result, fixing issues, asking for feedback and making sure the work is useful to the person who needs it.

I am a fresher, so I will need guidance, but I am ready to take initiative and grow into more responsibility. I would start by understanding the team's needs, suggesting ideas that can be tested and following through on the work.